# DLP Lab6: Conditional Generative Models

Yi-Hsiang Ho, 111550106

## 1. Introduction

In this lab, I implemented and evaluated a conditional generative model for multi-label image synthesis using the **iclevr** dataset. The core objective is to generate $64 \times 64$ images that accurately reflect multiple object constraints (e.g., color and shape) through a denoising process.

The implementation leverages several key technologies to achieve high-fidelity conditional generation:

- **DDPM / DDIM**: Denoising Diffusion Probabilistic Models for the generative backbone, with Denoising Diffusion Implicit Models used for efficient sampling.
- **Cross-Attention Conditioning**: A mechanism to integrate multi-label embeddings into the U-Net architecture, allowing for precise spatial control.
- **Classifier-Free Guidance (CFG)**: A technique to enhance the alignment between the generated images and the input conditions.

## 2. Implementation Details

### 2.1. Model Architecture

The core of the generative model is a **Class-Conditioned U-Net** implemented using the `diffusers` library's `UNet2DConditionModel`. The architecture consists of several downsampling and upsampling blocks with skip connections:

- **Downsampling**: Two `DownBlock2D` followed by two `CrossAttnDownBlock2D`.
- **Upsampling**: Two `CrossAttnUpBlock2D` followed by two `UpBlock2D`.
- **Channels**: The block output channels are (64, 128, 256, 512).

I utilized **Cross-Attention** conditioning for the final model. In this setup, the class labels are passed as `encoder_hidden_states` to the U-Net. This allows the model to spatially attend to specific object requirements during the denoising process. The code snippet is in Fig. 1.

```python
class ClassConditionedUnet(nn.Module):
    def __init__(self, args):
        super().__init__()
        self.cond_type = args.cond_type
        # The embedding layer
        self.class_emb = nn.Linear(args.num_classes, args.class_emb_size)

        if self.cond_type == "concat":  # Concat condition embedding with image
            self.model = UNet2DModel(
                sample_size=args.image_size,
                in_channels=3 + args.class_emb_size,
                out_channels=3,
                layers_per_block=2,
                block_out_channels=(64, 128, 256, 512),
                down_block_types=(
                    "DownBlock2D",        # a regular ResNet downsampling block
                    "DownBlock2D",
                    "AttnDownBlock2D",    # a ResNet downsampling block with spatial self-attention
                    "AttnDownBlock2D",
                ),
                up_block_types=(
                    "AttnUpBlock2D",
                    "AttnUpBlock2D",
                    "UpBlock2D",
                    "UpBlock2D",
                ),
            )
        else:  # Cross-Attention conditioning
            self.model = UNet2DConditionModel(
                sample_size=args.image_size,
                in_channels=3,
                out_channels=3,
                layers_per_block=2,
                block_out_channels=(64, 128, 256, 512),
                down_block_types=(
                    "DownBlock2D",
                    "DownBlock2D",
                    "CrossAttnDownBlock2D",
                    "CrossAttnDownBlock2D",
                ),
                up_block_types=(
                    "CrossAttnUpBlock2D",
                    "CrossAttnUpBlock2D",
                    "UpBlock2D",
                    "UpBlock2D",
                ),
                cross_attention_dim=args.class_emb_size,  # Cross attention with embedding
            )
```

Figure 1. **Model architecture**.

### 2.2. Conditioning Mechanism

The input labels are multi-hot vectors of size 24 (representing colors and shapes). These labels are processed as follows and the implementation is in Fig. 2:

1. The multi-hot label $L \in \{0, 1\}^{24}$ is projected through a `nn.Linear` layer to a 256-dimensional embedding space.
2. For Cross-Attention, this 256D vector is unsqueezed to form a sequence of length 1, acting as the context for the attention mechanism.
3. For the Concatenation baseline, the vector is reshaped to $(256, 1, 1)$ and expanded to match the spatial resolution of the image $(256, 64, 64)$ before being concatenated with the noisy image input.

```python
class ClassCondUnet(nn.Module):
    def __init__(self, args):
        # ...
    def forward(self, x, t, class_labels):
        b, c, w, h = x.shape

        class_cond = self.class_emb(class_labels.float())  # (b, class_emb_size)

        if self.cond_type == "concat":
            class_cond_reshape = class_cond.view(b, class_cond.shape[1], 1, 1).expand(b, class_cond.shape[1], w, h)
            # x: (b, c, w, h), class_cond: (b, class_emb_size, w, h)
            net_input = torch.cat((x, class_cond_reshape), 1)  # (b, c + class_emb_size, w, h)
            return self.model(net_input, t).sample  # (b, c, w, h)
        else:
            # encoder_hidden_states: (b, seq_len=1, feat_dim=class_emb_size)
            return self.model(x, t, encoder_hidden_states=class_cond.unsqueeze(1)).sample
```

Figure 2. **Conditioning Mechanism**.

### 2.3. Training Procedure

The model was trained for 200 epochs using the following hyperparameters:

- **Optimizer**: AdamW with a learning rate of $10^{-4}$ and weight decay of 0.01.
- **Learning Rate Scheduler**: Cosine Annealing with $T_{max} = 200$.
- **Batch Size**: 64.
- **Loss Function**: Mean Squared Error (MSE) between the predicted noise $\hat{\epsilon}$ and the actual noise $\epsilon$.
- **Noise Schedule**: 1000 training timesteps with a `scaled_linear` beta schedule.

To enable Classifier-Free Guidance (CFG), I implemented a joint training strategy where the model learns both conditional and unconditional distributions.

### 2.4. Classifier-Free Guidance (CFG)

To enhance the model's adherence to the multi-label conditions, I implemented Classifier-Free Guidance (CFG). During training, I applied a 20% probability of setting the conditioning labels to a null vector (zero-hot).

During inference, the final noise prediction $\hat{\epsilon}_\theta$ is calculated by linearly extrapolating from the unconditional prediction $\epsilon_\theta(x_t, \emptyset)$ towards the conditional prediction $\epsilon_\theta(x_t, c)$ using a guidance scale $w$:

$$\hat{\epsilon}_\theta(x_t, c) = \epsilon_\theta(x_t, \emptyset) + w \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \emptyset)) \quad (1)$$

The sampling implementation is illustrated in Fig. 3.

```python
sample = torch.randn(batch_size, 3, args.image_size, args.image_size).to(device)
uncond_labels = torch.zeros_like(labels).to(device)

for i, t in enumerate(tqdm(scheduler.timesteps, desc=f"Sampling {save_subdir}", leave=False)):
    if args.use_cfg:
        batched_sample = torch.cat([sample, sample], dim=0)
        batched_t = torch.full((2 * batch_size,), t.item(), device=device, dtype=torch.long) if t.ndim == 0 else torch.cat([t, t], dim=0).to(device)
        batched_labels = torch.cat([labels, uncond_labels], dim=0)

        model_output = model(batched_sample, batched_t, batched_labels)
        cond_res, uncond_res = torch.chunk(model_output, 2, dim=0)
        residual = uncond_res + args.cfg_weight * (cond_res - uncond_res)
    else:
        residual = model(sample, t, labels)

    sample = scheduler.step(residual, t, sample).prev_sample

    if b_idx == 0 and i in capture_indices:
        denoising_history.append(sample[:5].detach().cpu())
```

Figure 3. **Classifier-Free Guidance**.

### 2.5. Denoising Diffusion Implicit Models (DDIM)

For accelerated inference, I introduced Denoising Diffusion Implicit Models (DDIM). Unlike the standard DDPM which requires 1000 sequential steps, DDIM defines a non-Markovian forward process that allows for high-quality generation in significantly fewer steps (e.g., 50–100 steps) by following a deterministic sampling trajectory. The detailed implementation involves replacing the noise scheduler from `DDPMScheduler` to `DDIMScheduler`, both of which are provided by the `diffusers` library.

While DDIM offers a nearly 50× speedup, I observed that it occasionally resulted in slightly lower classification accuracy compared to the full DDPM schedule for complex multi-label scenes. Consequently, while DDIM is supported for fast prototyping, the results reported in Sec. 3 were obtained using 1000-step DDPM to ensure maximum fidelity.

## 3. Results and Discussion

### 3.1. Synthetic Image Grids

The generated images for `test.json` and `new_test.json` are shown in Fig. 4. The model successfully generates multiple objects with correct colors and shapes.

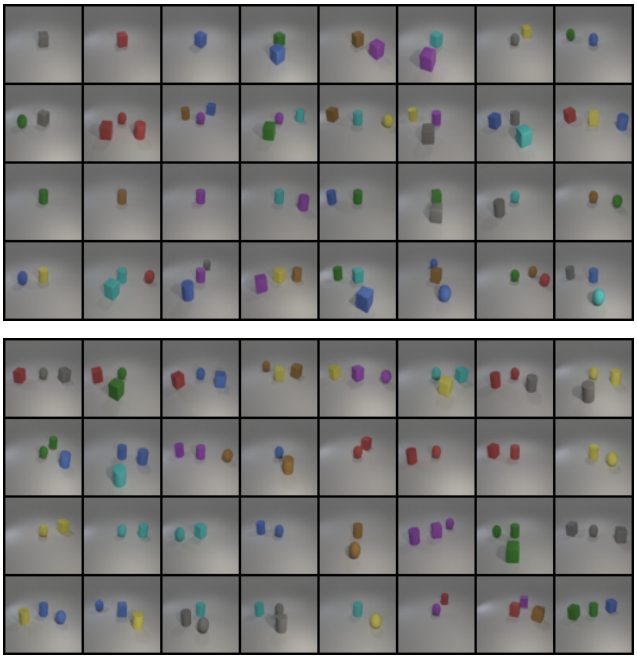


Figure 4. **Synthetic Image Grids**. Up: `test.json`, Down: `new_test.json`.

### 3.2. Denoising Process

The denoising process for the label set [”red sphere”, ”cyan cylinder”, ”cyan cube”] is shown in Fig. 5. The objects gradually emerge from Gaussian noise as the timesteps decrease.

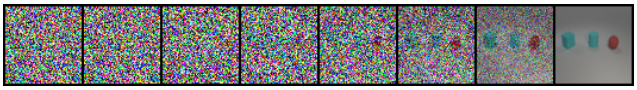


Figure 5. **Denoising Process** for [red sphere, cyan cylinder, cyan cube].

### 3.3. Classification Accuracy

The classification accuracy was evaluated using a pre-trained evaluation model. These results were achieved using DDPM with 1000 steps and a CFG weight of 3.0.

| Test Set | Accuracy | Inference Time |
|---|---|---|
| `test.json` | 0.8611 | 59.47s |
| `new_test.json` | 0.9048 | 59.68s |

Table 1. **Classification Accuracy** on test data.

The accuracy screenshots are provided in Fig. 6.




Figure 6. **Accuracy Screenshots**.

### 3.4. Discussion of Experiments

#### Conditioning Type

I compared **Concatenation** and **Cross-Attention** for multi-label conditioning. While Concatenation struggled to resolve spatial constraints (approx. 0.43 accuracy), Cross-Attention allowed the U-Net to attend to labels at multiple scales, achieving significantly higher accuracy (over 0.86). Results are summarized in Tab. 2.

| Conditioning Type | Test Acc | New Test Acc |
|---|---|---|
| Concatenation | 0.4306 | 0.5595 |
| Cross-Attention | **0.8611** | **0.9048** |

Table 2. **Ablation Study: Conditioning Mechanism** (DDPM, 1000 steps, CFG: 3.0, Beta: scaled linear). The highest accuracy is marked in bold.

#### Classifier-Free Guidance (CFG) Weight

The guidance scale $w$ balances fidelity and diversity. These experiments used DDIM with 50 steps to analyze the trend (Tab. 3).

- $w = 1.0$: Results in lower clarity and weaker alignment with input labels.
- $w = 2.0$: Optimal balance for conditional fidelity and image quality.
- $w \geq 5.0$: Causes "guidance collapse," where oversaturation and shape distortion significantly degrade scores. Notably, when $w > 8$, the model begins to ignore the input conditions entirely as the guidance signal overpowers the generative process.

Visual comparisons are in Fig. 7.

| CFG Weight ($w$) | Test Acc | New Test Acc |
|---|---|---|
| w/o CFG | 0.7639 | 0.8571 |
| 1.0 | 0.6806 | 0.8333 |
| 2.0 | **0.7778** | **0.8929** |
| 3.0 | **0.7778** | 0.8452 |
| 5.0 | 0.6944 | 0.5952 |
| 8.0 | 0.2917 | 0.4167 |

Table 3. **Ablation Study: Classifier-Free Guidance Weight** (DDIM, 50 steps, Beta: scaled linear). The highest accuracy is marked in bold.

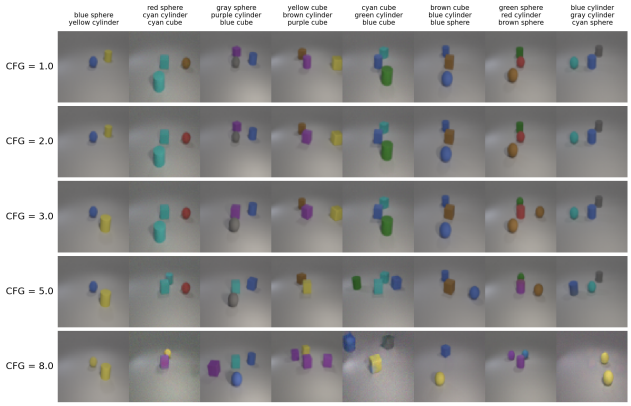



Figure 7. **Classifier-Free Guidance Weight**.

#### Beta Schedule Analysis

The noise schedule dictates the pace of information retrieval (Tab. 4). While **Linear** destroys structure too rapidly, **Scaled Linear** and **Squared Cosine** provide a more gradual transition. Squared Cosine achieved the highest accuracy as it preserves structural information more effectively during the early stages of denoising.

| Beta Schedule | Test Acc | New Test Acc |
|---|---|---|
| Linear | 0.6944 | 0.8095 |
| Scaled Linear | 0.7639 | 0.8095 |
| Squared Cosine | **0.7917** | **0.8333** |

Table 4. **Ablation Study: Beta Schedule** (DDIM, 50 steps, CFG=3.0). The highest accuracy is marked in bold.

### DDPM vs. DDIM Sampling

We evaluated the speed-accuracy trade-off between DDPM and DDIM (Tab. 5).

- **DDPM (1000 steps)**: Achieved the peak performance with **0.8611** and **0.9048** accuracy. The 1000-step trajectory provides superior stability for resolving multi-label spatial constraints, though it requires significantly more time ($\sim$119s).
- **DDIM (10–100 steps)**: Offers a substantial speedup, with 10 steps taking only 2.48s. While efficient, it results in lower accuracy. Notably, increasing DDIM steps beyond 50 did not improve results, suggesting that the model benefits from the stochastic nature of DDPM for fine error correction.

| Scheduler | Steps | Test Acc | New Test Acc | Total Time |
|---|---|---|---|---|
| DDIM | 10 | 0.7778 | 0.8452 | 2.48s |
| DDIM | 25 | 0.7778 | 0.8571 | 4.41s |
| DDIM | 50 | 0.7778 | 0.8452 | 7.14s |
| DDIM | 100 | 0.7222 | 0.8452 | 12.99s |
| DDPM | 1000 | **0.8611** | **0.9048** | 119.20s |

Table 5. **Ablation Study: Sampling Steps and Scheduler** (CFG: 3.0, Beta: scaled linear). The highest accuracy is marked in bold.

## 4. Conclusion

In this lab, I successfully implemented a conditional DDPM/DDIM model for multi-label image generation. By utilizing Cross-Attention and Classifier-Free Guidance, the model achieved high classification accuracy and produced visually plausible images on the iclevr dataset. The experiments highlighted the importance of conditioning mechanisms and guidance weights in controlling the generative process.